

5. Préparation de l'environnement

5.1. Créer le dossier du projet

```
mkdir tp_chapitre2_pipeline
```

```
cd tp_chapitre2_pipeline
```

5.2. Créer et activer l'environnement virtuel

```
python3 -m venv .venv
```

```
PS C:\Users\LI\my_dataops_tp> python -m venv .venv
Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File "C:\Users\LI\AppData\Local\Programs\Python\Python311\Lib\venv\__main__.py", line 6, in <module>
    main()
  File "C:\Users\LI\AppData\Local\Programs\Python\Python311\Lib\venv\__init__.py", line 546, in main
    builder.create(d)
  File "C:\Users\LI\AppData\Local\Programs\Python\Python311\Lib\venv\__init__.py", line 76, in create
    self._setup_pip(context)
  File "C:\Users\LI\AppData\Local\Programs\Python\Python311\Lib\venv\__init__.py", line 358, in _setup_pip
    self._call_new_python(context, '-m', 'ensurepip', '--upgrade',
  File "C:\Users\LI\AppData\Local\Programs\Python\Python311\Lib\venv\__init__.py", line 354, in _call_new_python
```

```
source .venv/bin/activate
```

```
KeyboardInterrupt
PS C:\Users\LI\my_dataops_tp> .venv\Scripts\activate
```

5.3. Installer les dépendances

```
(.venv) PS C:\Users\LI\my_dataops_tp> pip install -r requirements.txt
Collecting pandas (from -r requirements.txt (line 1))
  Using cached pandas-3.0.3-cp311-cp311-win_amd64.whl.metadata (19 kB)
Collecting duckdb (from -r requirements.txt (line 2))
  Using cached duckdb-1.5.3-cp311-cp311-win_amd64.whl.metadata (4.2 kB)
Collecting dlt (from -r requirements.txt (line 3))
  Downloading dlt-1.27.2-py3-none-any.whl.metadata (15 kB)
Collecting dbt-duckdb (from -r requirements.txt (line 4))
  Using cached dbt_duckdb-1.10.1-py3-none-any.whl.metadata (38 kB)
Collecting dagster (from -r requirements.txt (line 5))
```

6. Initialisation Git

6.1. Initialiser le dépôt

```
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\LI\my_dataops_tp> git init
(.venv) PS C:\Users\LI\my_dataops_tp> git config --global init.defaultBranch main
(.venv) PS C:\Users\LI\my_dataops_tp> git add .
```

6.2. Créer le fichier .gitignore

```

❖ .gitignore
1  .venv/
2  __pycache__/
3  target/
4  dbt_packages/
5  *.duckdb
```

6.3. Premier commit

```
(.venv) PS C:\Users\LI\my_dataops_tp> git add .
(.venv) PS C:\Users\LI\my_dataops_tp> git commit -m "Initialisation
[master (root-commit) 0bcb823] Initialisation du projet pipeline
2 files changed, 11 insertions(+)
create mode 100644 .gitignore
create mode 100644 requirements.txt
```

7.

Créer l'arborescence

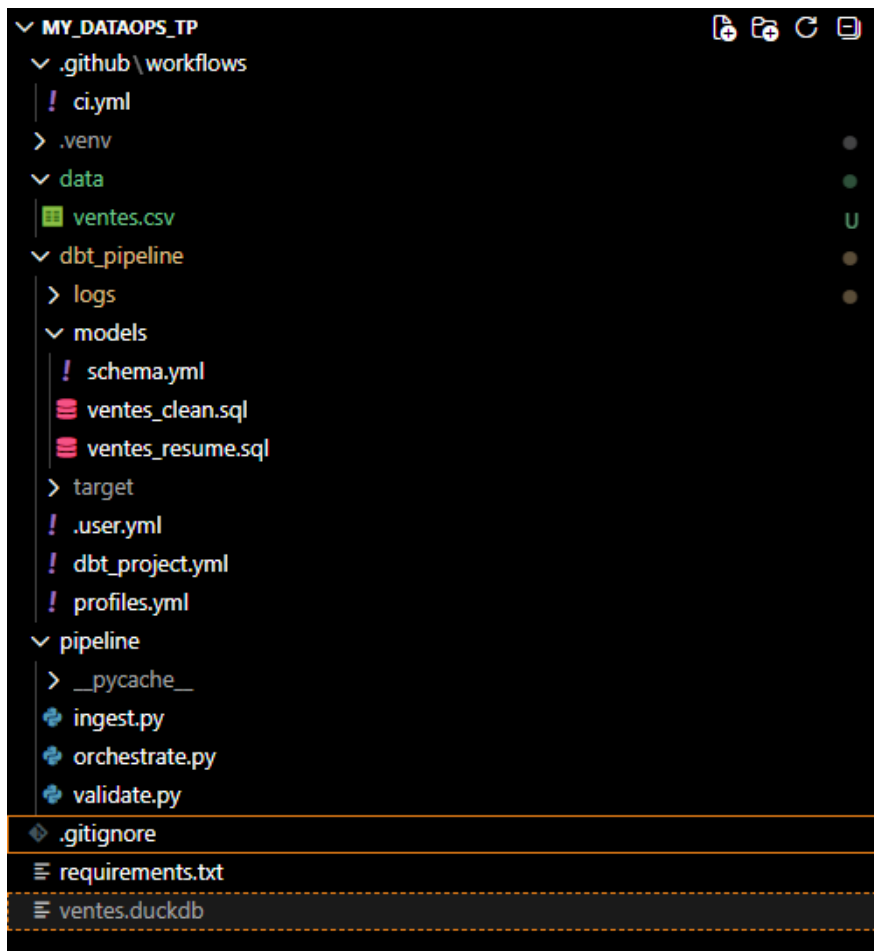
```
(.venv) PS C:\Users\LI\my_dataops_tp> mkdir data, pipeline, dbt_pipe
.github, .github/workflows
```

Directory: C:\Users\LI\my_dataops_tp

Mode	LastWriteTime	Length	Name
d-----	5/31/2026 3:27 PM		data
d-----	5/31/2026 3:27 PM		pipeline
d-----	5/31/2026 3:27 PM		dbt_pipeline

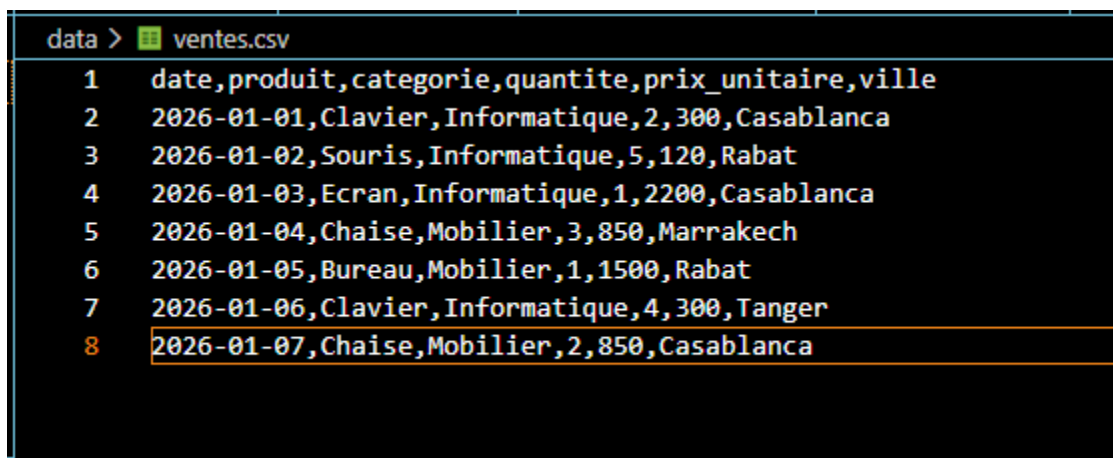
Directory: C:\Users\LI\my_dataops_tp\dbt_pipeline

Mode	LastWriteTime	Length	Name
d-----	5/31/2026 3:27 PM		models



8. Créer le fichier source

Créer data/ventes.csv



9. Étape 1 — Ingestion

Créer pipeline/ingest.py

```
(.venv) PS C:\Users\LI\my_dataops_tp> python pipeline/ingest.py
● >>
Traceback (most recent call last):
  File "C:\Users\LI\my_dataops_tp\pipeline\ingest.py", line 8, in <m
    df = pd.read_csv(csv_path)
    ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\LI\my_dataops_tp\.venv\Lib\site-packages\pandas\io\
873, in read_csv
    return _read(filepath_or_buffer, kwds)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\LI\my_dataops_tp\.venv\Lib\site-packages\pandas\io\
300, in _read
    parser = TextFileReader(filepath_or_buffer, **kwds)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Commit

```
(.venv) PS C:\Users\LI\my_dataops_tp> git add pipeline/ingest.py
(.venv) PS C:\Users\LI\my_dataops_tp> git commit -m "Ajout de l'étap
[master 4f8b45c] Ajout de l'étape d'ingestion
1 file changed, 12 insertions(+)
○ create mode 100644 pipeline/ingest.py
```

10. Étape 2 — Validation minimale

Créer pipeline/validate.py

```
pipeline > validate.py > ...
1  import duckdb
2
3  db_path = "ventes.duckdb"
4  # These are the columns we expect to find
5  required_columns = {"date", "produit", "categorie", "quantite", "prix_unitaire"}
6
7  con = duckdb.connect(db_path)
8
9  # Check if the table exists and get columns
10 columns = con.execute("DESCRIBE ventes_raw").fetchall()
11 existing_columns = {col[0] for col in columns}
12
13 # Check for missing columns
```

Exécuter

```
(.venv) PS C:\Users\LI\my_dataops_tp> python pipeline/validate.py
>>
Validation réussie : schéma et qualité minimale OK
```

Commit

```
(.venv) PS C:\Users\LI\my_dataops_tp> git add pipeline/validate.py
(.venv) PS C:\Users\LI\my_dataops_tp> git commit -m "Ajout de l'étape de validation minimale"
[master f334984] Ajout de l'étape de validation minimale
1 file changed, 30 insertions(+)
create mode 100644 pipeline/validate.py
```

11. Étape 3 — Transformation avec dbt

Créer dbt_pipeline/dbt_project.yml

```

dbt_pipeline > ! dbt_project.yml
1
2   name: 'dbt_pipeline'
3   version: '1.0.0'
4   config-version: 2
5   profile: 'dbt_pipeline'
6   model-paths: ["models"]
7
8   models:
9     dbt_pipeline:
10      +materialized: table
11

```

Créer dbt_pipeline/profiles.yml

```

dbt_pipeline > ! profiles.yml
1   dbt_pipeline:
2     target: dev
3     outputs:
4       dev:
5         type: duckdb
6         path: ../ventes.duckdb
7         threads: 1

```

Créer dbt_pipeline/models/ventes_clean.sql

```

dbt_pipeline > models > ventes_clean.sql
1   select
2     date,
3     produit,
4     categorie,
5     quantite,
6     prix_unitaire,
7     ville,
8     (quantite * prix_unitaire) as chiffre_affaires
9   from ventes_raw
10  where quantite > 0 and prix_unitaire > 0

```

Créer dbt_pipeline/models/ventes_resume.sql

```
dbt_pipeline > models >  ventes_resume.sql

1  select
2      categorie,
3      sum(quantite) as total_quantite,
4      sum(chiffre_affaires) as total_ca
5  from {{ ref('ventes_clean') }}
6  group by categorie
```

Exécuter

cd dbt_pipeline

```
(.venv) PS C:\Users\LI\my_dataops_tp\dbt_pipeline> dbt debug --profiles-dir .
16:29:50 Running with dbt=1.11.11
16:29:50 dbt version: 1.11.11
16:29:50 python version: 3.11.9
16:29:50 python path: C:\Users\LI\my_dataops_tp\.venv\Scripts\python.exe
16:29:50 os info: Windows-10-10.0.19045-SP0
16:29:51 Using profiles dir at .
16:29:51 Using profiles.yml file at .\profiles.yml
16:29:51 Using dbt_project.yml file at C:\Users\LI\my_dataops_tp\dbt_pipeline\dbt_project.yml
16:29:51 adapter type: duckdb
16:29:51 adapter version: 1.10.1
16:29:52 Configuration:
16:29:52   profiles.yml file [OK found and valid]
16:29:52   dbt_project.yml file [OK found and valid]
16:29:52 Required dependencies:
16:29:53   - git [OK found]

(.venv) PS C:\Users\LI\my_dataops_tp\dbt_pipeline> dbt run --profiles-dir .
16:54:22 Running with dbt=1.11.11
16:54:23 Registered adapter: duckdb=1.10.1
16:54:24 Unable to do partial parsing because saved manifest not found. Starting full parse.
16:54:30 Found 2 models, 485 macros
16:54:30
16:54:30 Concurrency: 1 threads (target='dev')
16:54:31 1 of 2 START sql table model main.ventes_clean ..... [RUN]
16:54:32 1 of 2 OK created sql table model main.ventes_clean ..... [OK in 0.
16:54:32 2 of 2 START sql table model main.ventes_resume ..... [RUN]
16:54:32 2 of 2 OK created sql table model main.ventes_resume ..... [OK in 0.
16:54:32
16:54:32 Finished running 2 table models in 0 hours 0 minutes and 2.26 seconds (2.26s).
16:54:33
16:54:33 Completed successfully
16:54:33
16:54:33 Done. PASS=2 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=2
(.venv) PS C:\Users\LI\my_dataops_tp\dbt_pipeline> cd ..
(.venv) PS C:\Users\LI\my_dataops_tp> |
```

Commit

git add dbt_pipeline

```
(.venv) PS C:\Users\LI\my_dataops_tp> git add dbt_pipeline
```

git commit -m "Ajout des modèles dbt"

```
(.venv) PS C:\Users\LI\my_dataops_tp> git commit -m "Ajout des modèles dbt"
[master 5733cc7] Ajout des modèles dbt
 6 files changed, 358 insertions(+)
 create mode 100644 dbt_pipeline/.user.yml
 create mode 100644 dbt_pipeline/dbt_project.yml
 create mode 100644 dbt_pipeline/logs/dbt.log
 create mode 100644 dbt_pipeline/models/ventes_clean.sql
 create mode 100644 dbt_pipeline/models/ventes_resume.sql
 create mode 100644 dbt_pipeline/profiles.yml
(.venv) PS C:\Users\LI\my_dataops_tp>
```

12. Étape 4 — Test simple avec dbt

Créer dbt_pipeline/models/schema.yml

```
dbt_pipeline > models > ! schema.yml
1  version: 2
2
3  models:
4    - name: ventes_clean
5      columns:
6        - name: produit
7          tests:
8            - not_null
9        - name: quantite
10         tests:
11           - not_null
12
13    - name: ventes_resume
14      columns:
15        - name: categorie
16          tests:
17            - not_null
```

Exécuter

cd dbt_pipeline

dbt test --profiles-dir .

```
(.venv) PS C:\Users\LI\my_dataops_tp\dbt_pipeline> dbt test --profiles-dir .
17:15:28 Running with dbt=1.11.11
17:15:29 Registered adapter: duckdb=1.10.1
17:15:30 Found 2 models, 3 data tests, 485 macros
17:15:30
17:15:30 Concurrency: 1 threads (target='dev')
17:15:30
17:15:30 1 of 3 START test not_null_ventes_clean_produit ..... [RUN]
17:15:30 1 of 3 PASS not_null_ventes_clean_produit ..... [PASS in 0.08s]
17:15:30 2 of 3 START test not null ventes clean quantite ..... [RUN]
```

cd ..

Commit

git add dbt_pipeline/models/schema.yml

```
• (.venv) PS C:\Users\LI\my_dataops_tp> git add dbt_pipeline/models/schema.yml
• (.venv) PS C:\Users\LI\my_dataops_tp>
```

git commit -m "Ajout des tests simples dbt"

```
• (.venv) PS C:\Users\LI\my_dataops_tp> git commit -m "Ajout des tests simples dbt"
[master b7589c0] Ajout des tests simples dbt
 1 file changed, 17 insertions(+)
 create mode 100644 dbt_pipeline/models/schema.yml
• (.venv) PS C:\Users\LI\my_dataops_tp>
```

13. Étape 5 — Orchestration avec Dagster

Créer pipeline/orchestrate.py

```
pipeline > 📄 orchestrate.py > 📁 ventes_pipeline
1  from dagster import job, op
2  import os
3
4  @op
5  def ingest():
6      """Runs the ingestion script"""
7      return_code = os.system("python pipeline/ingest.py")
8      if return_code != 0:
9          raise Exception("Ingestion failed")
10
11  @op
12  def validate(ingest_done):
13      """Runs the validation script"""
14      return_code = os.system("python pipeline/validate.py")
15      if return_code != 0:
```

Lancer Dagster

dagster dev -f pipeline/orchestrate.py

```
• (.venv) PS C:\Users\LI\my_dataops_tp> dagster dev -f pipeline/orchestrate.py
C:\Users\LI\my_dataops_tp\venv\Lib\site-packages\click\core.py:877: SupersessionWarning: Function `dev_command` is superseded and its usage is discouraged. Use 'dg dev' instead.
  return callback(*args, **kwargs)
2026-05-31 18:42:43 +0100 - dagster - INFO - Using temporary directory C:\Users\LI\my_dataops_tp\.tmp_dagster_home_d83czu0a for storage. This will be removed when dagster dev exits.
2026-05-31 18:42:43 +0100 - dagster - INFO - To persist information across sessions, set the environment variable DAGSTER_HOME to a directory to use.
2026-05-31 18:42:47 +0100 - dagster - INFO - Launching Dagster services...
2026-05-31 18:43:02 +0100 - dagster.daemon - INFO - Instance is configured with the following daemons: ['AssetDaemon', 'BackfillDaemon', 'FreshnessDaemon', 'QueuedRunCoordinatorDaemon', 'SchedulerDaemon', 'SensorDaemon']
```

Commit

```
git add pipeline/orchestrate.py
```

```
git commit -m "Ajout de l'orchestration Dagster"
```

```
(.venv) PS C:\Users\LI\my_dataops_tp> git add pipeline/orchestrate.py
(.venv) PS C:\Users\LI\my_dataops_tp> git commit -m "Ajout de l'orchestration Dagster"
[master 438cf1d] Ajout de l'orchestration Dagster
1 file changed, 36 insertions(+)
create mode 100644 pipeline/orchestrate.py
(.venv) PS C:\Users\LI\my_dataops_tp>
```

14. Étape 6 — CI simple

Créer .github/workflows/ci.yml

```
.github > workflows > ! ci.yml
1  name: CI Pipeline
2
3  on:
4    push:
5      branches: [ master, main ]
6    pull_request:
7      branches: [ master, main ]
8
9  jobs:
10   validate-project:
11     runs-on: ubuntu-latest
12
13     steps:
14       - name: Checkout repository
15         uses: actions/checkout@v4
16
17       - name: Setup Python
18         uses: actions/setup-python@v5
19         with:
20           python-version: '3.11'
```

Commit

```
git add .github/workflows/ci.yml
```

```
git commit -m "Ajout d'une CI simple"
```

```
(.venv) PS C:\Users\LI\my_dataops_tp> git add .github/workflows/ci.yml
(.venv) PS C:\Users\LI\my_dataops_tp> git commit -m "Ajout d'une CI simple"
[master 33f28ac] Ajout d'une CI simple
1 file changed, 29 insertions(+)
create mode 100644 .github/workflows/ci.yml
(.venv) PS C:\Users\LI\my_dataops_tp>
```

1. En quoi ce pipeline est-il plus structuré qu'un notebook unique ?

Contrairement à un notebook qui est linéaire et souvent désordonné, ce pipeline est **modulaire**. Chaque étape (ingestion, validation, transformation) est isolée dans son propre fichier ou outil. Cela permet de tester chaque partie séparément, de faciliter la maintenance et d'éviter qu'une erreur dans une cellule ne bloque tout le reste de manière obscure.

2. Quel est le rôle de Git dans ce projet ?

Git assure le **versionnement et la traçabilité**. Il permet de garder un historique complet de l'évolution du code. Si une modification casse le pipeline, on peut revenir en arrière. Il permet aussi de documenter les étapes via les messages de "commit" (ex: "Ajout de l'étape de validation").

3. Quel est le rôle de requirements.txt ?

Il garantit la **reproductibilité technique**. En listant toutes les bibliothèques (pandas, dbt, dagster, etc.) et leurs versions, il permet à n'importe quel autre développeur (ou au serveur de CI) de recréer exactement le même environnement de travail avec une seule commande (pip install -r requirements.txt).

4. Pourquoi ajouter une étape validate.py ?

Cette étape sert de **"garde-fou" (Data Quality)**. Elle vérifie que les données brutes sont conformes (colonnes présentes, pas de valeurs nulles critiques) avant de lancer des transformations lourdes. Cela évite de propager des erreurs dans les rapports finaux ("Garbage In, Garbage Out").

5. Quel est le rôle de dbt test ?

Il permet d'automatiser les **tests de cohérence sur les données transformées**. Alors que validate.py vérifie l'entrée, dbt test vérifie la sortie (ex: vérifier que le chiffre d'affaires est bien calculé ou que les catégories ne sont pas vides). C'est un contrôle qualité continu.

6. Qu'apporte Dagster par rapport à une exécution manuelle ?

Dagster est l'**orchestrateur**. Il gère les dépendances entre les tâches : il sait qu'il ne doit pas lancer la transformation si la validation a échoué. Il offre aussi une interface visuelle pour surveiller l'exécution, relancer des étapes précises et voir les logs en cas d'erreur, ce qui est impossible avec des lancements manuels.

7. Qu'apporte la CI (GitHub Actions) ?

La CI (Intégration Continue) apporte une **automatisation des vérifications**. À chaque fois que vous "poussez" du code sur GitHub, le serveur vérifie automatiquement la syntaxe et les dépendances. Cela garantit que le projet reste fonctionnel et sain tout au long de son développement.

8. Que manque-t-il encore pour un pipeline pleinement industrialisé ?

Pour être prêt pour la production ("industrialisé"), il manque :

- **Le déploiement Cloud** : Le pipeline tourne ici en local ; il faudrait l'héberger (ex: AWS, Azure).
- **La gestion des secrets** : Stocker les mots de passe et accès aux bases de données de façon sécurisée (pas en clair).
- **Le monitoring/Alerting** : Envoyer un email ou un message Slack automatiquement si une étape échoue.
- **Le passage à l'échelle** : Gérer des millions de lignes de données au lieu d'un petit fichier CSV.

L'historique de Git

```
▼ TERMINAL
• (.venv) PS C:\Users\LI\my_dataops_tp> git log --oneline
33f28ac (HEAD -> master) Ajout d'une CI simple
438cf1d Ajout de l'orchestration Dagster
b7589c0 Ajout des tests simples dbt
5733cc7 Ajout des modèles dbt
f334984 Ajout de l'étape de validation minimale
4f8b45c Ajout de l'étape d'ingestion
0bcb823 Initialisation du projet pipeline
○ (.venv) PS C:\Users\LI\my_dataops_tp> |
```

L'historique de votre travail sur Git

